



Parametric liquidation cover for cross-chain lending positions, built on the Attestcoin Protocol.

Your position lives anywhere. Your protection lives here.

BUIDL CTC 2026 Fall — DeFi track

Live demo: <https://aniwere.vercel.app> — runs in labelled sample mode until the contracts are deployed; block heights and attestation status are always read live.

Indonesian version: [README.id.md](#)

What this is

A user buys protection on **Creditcoin** for their Aave V3 position on **Ethereum**. If that position is actually liquidated, the `LiquidationCall` event from Ethereum is proven cryptographically through the **Attestcoin Protocol**, and the payout is released automatically.

No manual claim. No committee. No centralized oracle. No bridge.

Honest status

Written here, at the top, so nobody has to guess.

STATUS	
Sepolia contract (<code>AniWereProbe</code>)	Done, 4/4 tests green
Creditcoin contracts (<code>AniWereASC</code> , <code>CoverVault</code> , <code>AttestcoinAdapter</code>)	Done, 12/12 tests green
Attestcoin verification path	Proven live. A proof from a real Sepolia transaction was accepted by the precompile; four corrupted proof variants all reverted
Off-chain proof worker	Done, and <code>status</code> has been run against the real endpoints
Frontend	Three pages, wired to chain through wagmi. Without a deployment it runs in a labelled sample mode — see the note below
Testnet deployment	Deployed 13 September — Sepolia probe + Creditcoin ASC, adapter, and vault. Addresses below

All contracts are live on testnet and their wiring was checked on-chain after deployment: the ASC points at the deployed probe and at Aave's Sepolia pool, the vault's controller is the ASC, and the adapter uses chain key 1 against the precompile at `0x...0FD2` . See [Deployed contracts](#).

The frontend has two modes, decided by a single env var:

- **Sample mode** (`NEXT_PUBLIC_ASC_ADDRESS` empty) — position and policy figures are illustrative, and every page carries a banner saying so. On-chain buttons are disabled.
- **Live mode** (env filled) — every number is read from the contracts through a connected wallet. The banner disappears. Probe, buy cover, and submit claim all work from the browser.

What is **always** real in both modes: the block heights in the header and the attestation panel in the Proof Explorer. Both are read from Sepolia and the Proof Builder on every request, with no wallet and no deployment.

What is actually proven

This is the part that gets misunderstood most often, so it is stated explicitly.

Attestcoin Readability proves that a transaction occurred in a source-chain block, through two proofs working together:

- **Continuity proof** — the block really is part of the Ethereum chain already attested on Creditcoin
- **Merkle proof** — the transaction really is inside that block

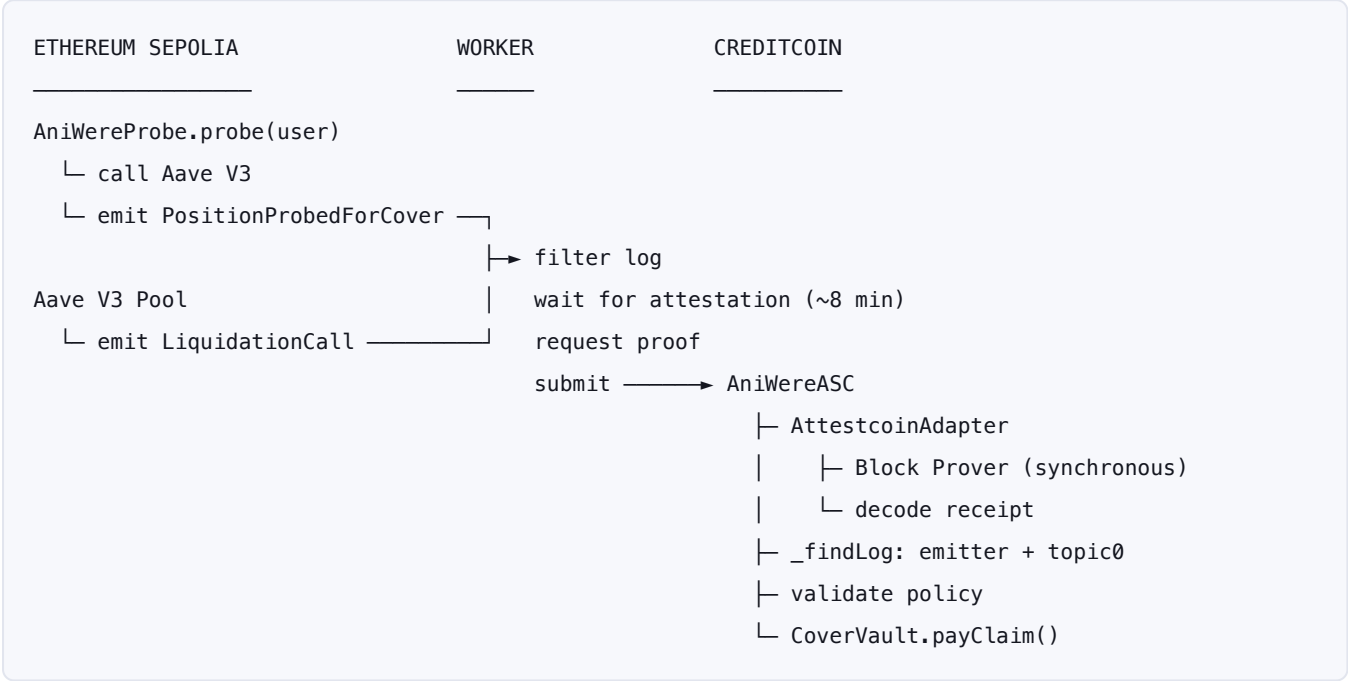
Both are verified **synchronously** by the Block Prover Precompile at `0x...0FD2` , so verification and payout happen inside the same Creditcoin transaction.

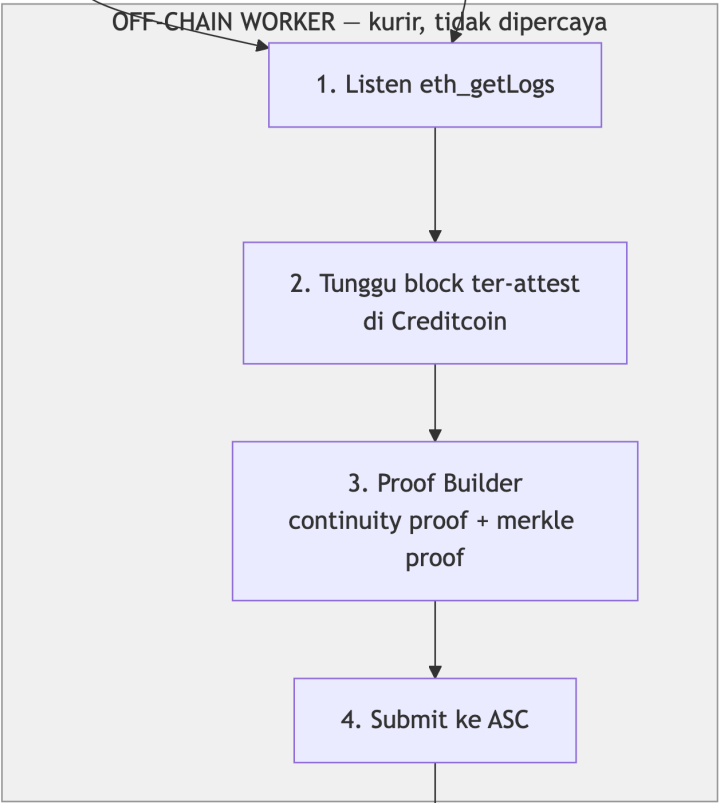
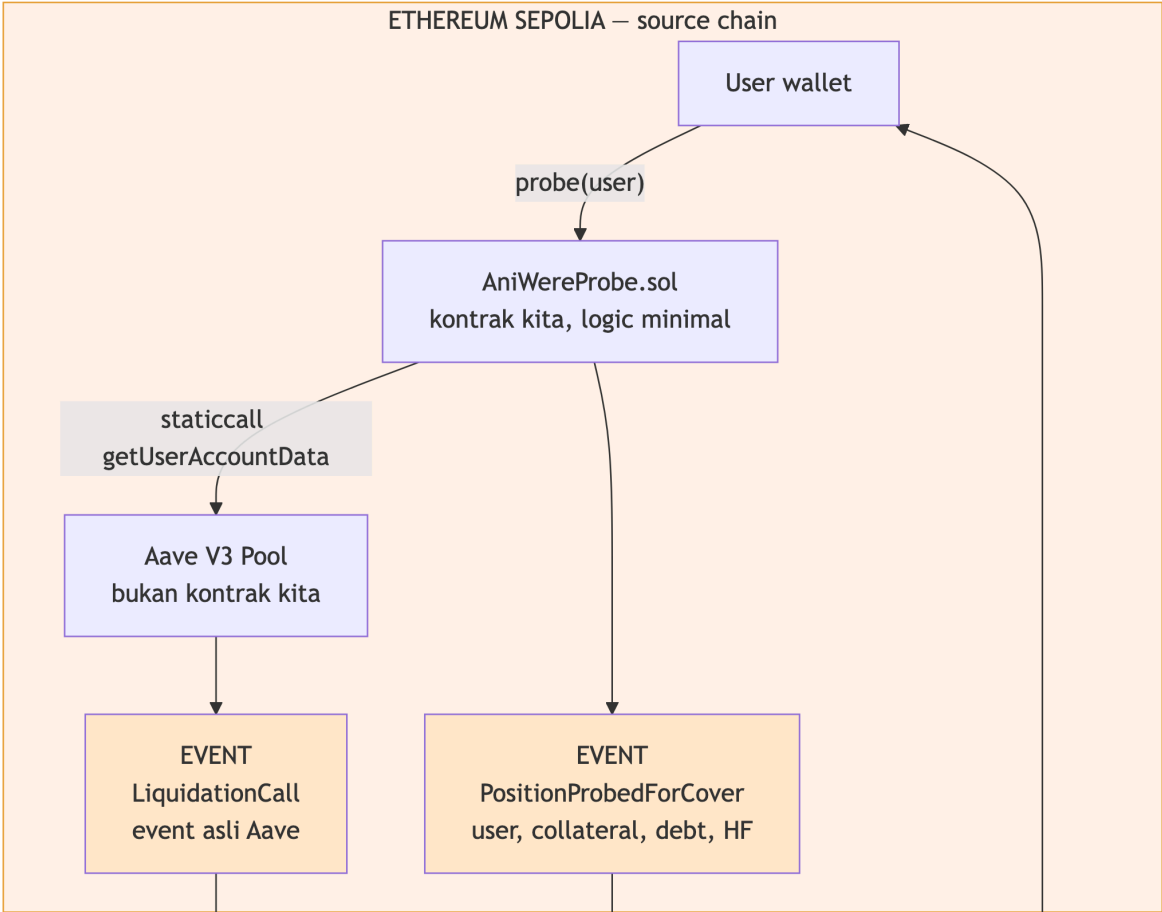
What can be proven: events in a transaction's receipt logs. **What CANNOT be proven:** storage slots.

As a consequence, `IPool.getUserAccountData(user)` cannot be proven directly — it is a view call against storage and leaves no trace in any receipt log. That is why we use the **prober pattern**: a contract on Sepolia calls Aave and emits the result as an event, and that event is what gets proven.

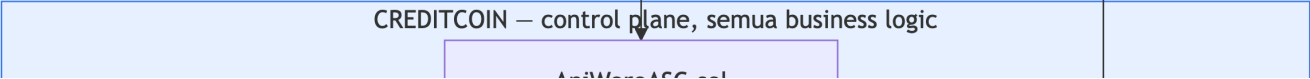
The precompile returns a `bool`, **not** the log. The receipt has to be decoded on the Creditcoin side. That is `AttestcoinAdapter` 's job, and all Attestcoin uncertainty stops in that file — `AniWereASC` knows nothing about the precompile.

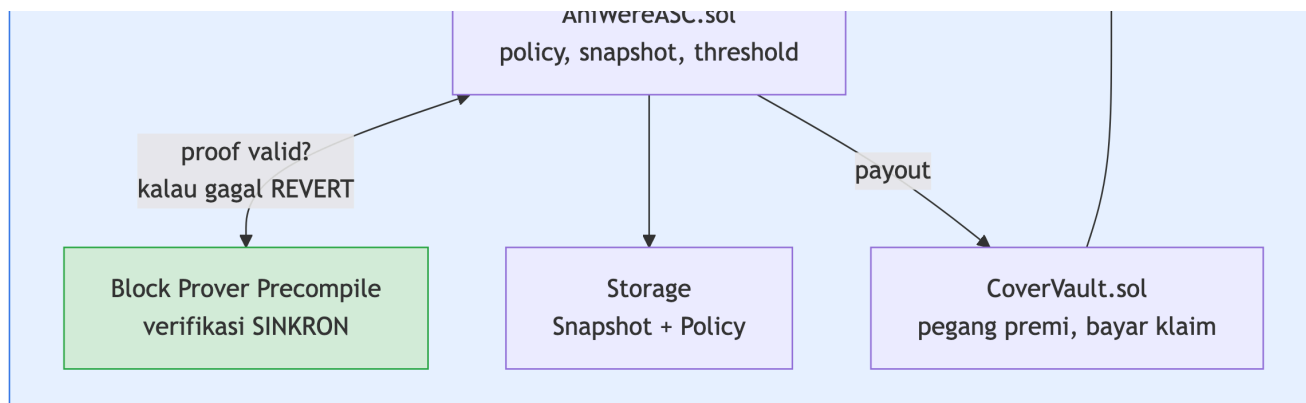
Architecture





CTC ke holder





Two detailed flows — buying cover, and claim/payout — exist as separate diagrams:

FLOW	DIAGRAM
Buy cover	2-flow-beli-cover.png
Claim & payout	3-flow-klaim-payout.png

The `.mermaid` sources live in `docs/diagrams/`; the PNGs are rendered with `mmdc -t default -b white -s 3`.

Design decisions

No privileged access

`CoverVault` is deployed **by** `AniWereASC` inside its constructor, so the ASC address stored in the vault is `immutable` and there is no setter at all.

- No `owner`, `onlyOwner`, `pause`, or upgrade path
- No withdrawal function for the team
- Funds leave through exactly two routes: a payout triggered by a valid proof, or an underwriter reclaiming capital that is not currently locked

The AniWere team technically cannot touch anyone's funds. If someone asks why they should trust us: they should not have to, and that is the whole point.

The worker is not trusted

The off-chain worker is only a courier. If it submits a forged proof, the precompile rejects it and the transaction reverts. It also submits the **entire receipt**, not a hand-picked log — log selection happens in `_findLog` inside the contract, so the worker never gets to choose on our behalf.

The only thing a malicious worker can do is not submit. And because `submitLiquidationClaim` is permissionless, anyone can take its place.

Solvency guard

Total active cover can never exceed the vault's free capital. The vault cannot become insolvent by construction.

Health factor is never recomputed

We use the number from `getUserAccountData` as-is, so the health factor we prove is identical to the one Aave uses to decide a liquidation. Recomputing it from another price source would immediately reintroduce the trust assumption we set out to remove.

The frontend is never a required path

All three on-chain actions in the UI — probe, submit position proof, submit liquidation claim — have a one-line worker equivalent, and the relevant page shows the command. This is not accidental duplication:

`submitPositionProof` and `submitLiquidationClaim` are deliberately permissionless, so a policy holder never depends on our frontend or our worker to get paid.

Proofs are fetched through our own server routes (`/api/attestcoin/*`) because the Proof Builder does not send CORS headers. Those routes grant no additional authority — a proof passing through them still has to satisfy the precompile inside the contract.

Emitter check

`_findLog` matches **both the emitter and topic0**, not topic0 alone. Without it, anyone could deploy a contract on Sepolia that emits an event with the same signature and drain the vault.

This is stricter than the official Attestcoin example, whose `getLogsByEventSignature` filters on topic0 only. There is a test for it: `test_RejectsLiquidationFromWrongEmitter` .

Anti-replay binds the block, not just the transaction

`proofId` is a hash of chainKey + height + merkle root + transaction contents. If it were only a hash of the raw transaction, the same transaction could be replayed through a different block. There is a test for it:

`test_SameTransactionAtDifferentHeightIsSeparateProof` .

Repository layout

```
aniwere/
├─ app/                Frontend (Next.js)
├─ contracts-sepolia/  Foundry – AniWereProbe.sol
├─ contracts-creditcoin/ Foundry – AniWereASC.sol, CoverVault.sol, AttestcoinAdapter.sol
├─ worker/            Off-chain proof worker (TypeScript + viem)
├─ video/             Demo video (Remotion + macOS TTS)
├─ docs/
│   ├─ PRD.md          Product context
│   ├─ ATTESTCOIN.md   Live validation notes – the second most important document after this one
│   ├─ DEMO.md          Deploy + demo runbook
│   ├─ SUBMISSION.md   DoraHacks submission draft
│   ├─ VIDEO-SCRIPT.md Screencast script
│   ├─ SPRINT.md        Daily plan
│   └─ diagrams/

```

Two separate Foundry projects, not one. Their network configs and EVM targets differ, and merging them makes it easy to deploy to the wrong chain.

Test results

```
contracts-sepolia      4 passed, 0 failed
contracts-creditcoin  12 passed, 0 failed
```

What the tests guard, rather than merely count:

TEST	WHAT IT PREVENTS
<code>test_RejectsLiquidationFromWrongEmitter</code>	A fake contract on Sepolia draining the vault
<code>test_RejectsReplayedProof</code>	One liquidation being paid twice
<code>test_SameTransactionAtDifferentHeightIsSeparateProof</code>	A <code>proofId</code> that is too weak
<code>test_CannotOversellCover</code>	The vault selling more protection than its capital
<code>test_UnderwriterCannotWithdrawLockedCapital</code>	Capital withdrawn while it still backs an active policy
<code>test_CannotBuyCoverWhenAlreadyUnhealthy</code>	Buying protection when liquidation is practically certain
<code>test_CannotBuyCoverWithStaleSnapshot</code>	Buying protection against stale position figures
<code>test_OnlyASCCanTouchVault</code>	Anyone but the ASC moving funds

All tests run against `MockProver`. The pallet-evm precompile has no bytecode, so Foundry fork tests cannot reach it — the real receipt-decoding path is only proven at testnet deployment.

Setup

```
# Source-chain side
cd contracts-sepolia
forge install foundry-rs/forge-std --no-git
cp .env.example .env      # fill it in first
forge build && forge test

# Creditcoin side
cd ../contracts-creditcoin
forge install foundry-rs/forge-std --no-git
cp .env.example .env      # fill it in first
forge build && forge test

# Worker
cd ../worker
npm install
cp .env.example .env
npm run status             # checks Sepolia, Proof Builder, Creditcoin. No wallet needed.
```

If this is already a git repo, drop `--no-git` so dependencies are added as submodules.

Deploy

```
# 1. Probe to Sepolia
cd contracts-sepolia
forge script script/DeployProbe.s.sol:DeployProbe --rpc-url sepolia --broadcast --verify

# 2. Put the address into contracts-creditcoin/.env as SOURCE_PROBE_SEPOLIA
#    and into worker/.env as PROBE_ADDRESS

# 3. ASC to Creditcoin
cd ../contracts-creditcoin
forge script script/DeployASC.s.sol:DeployASC --rpc-url creditcoin_testnet --broadcast --legacy

# 4. Fund the vault before the demo
cast send <vault> "depositCapital()" --value 500ether --rpc-url creditcoin_testnet --private-key $PRIV
```

`EvmV1Decoder` is a library with public functions, so forge deploys and links it automatically. If gas estimation fails, add `--gas-estimate-multiplier 135`.

Running the full flow

```
cd worker
npm run worker -- status           # everything green first
npm run worker -- probe 0xUSER     # emit the event on Sepolia
npm run worker -- submit-position 0xTX # wait for attestation, prove, store on Creditcoin
npm run worker -- watch            # loop: watch probes + liquidations
```

Worker details are in `worker/README.md` .

The full runbook, from an empty wallet through to a finished recording — including what to do if a liquidation cannot be triggered or the precompile rejects — is in `docs/DEMO.md` .

Latency expectations

Do not use the phrase "real-time".

STAGE	MEASURED
Source block lands on Sepolia	0
Attested on Creditcoin	~35–40 blocks, ~8 minutes
Proof verification + payout	one Creditcoin block (~15 seconds)
Total	~8 minutes

The correct framing:

Verified one Creditcoin block after the source block is attested — about 8 minutes after it lands on Ethereum.

Four independent measurements (27 August, 4 September, 8 September, 13 September) using different instruments. The lag behind head is practically identical every time. Details in `docs/ATTESTCOIN.md` section 4.

We do **not** use the sentence "*within one Creditcoin block of Ethereum finality*", however well it reads. Attestation measurably runs 20–54 blocks **ahead of finalized** , so that sentence would promise a guarantee the protocol does not actually give. The consequence is in the limitations list below, item 2.

Limitations we acknowledge

1. **Snapshots, not a stream.** A verified health factor is true at a specific block. Between two probes we know nothing. This is a direct consequence of the event-proof model.
2. **Attestation runs ahead of Ethereum finality.** A proof can pass for a block that is theoretically still reorgable. A depth of 20–54 blocks is far outside any plausible Ethereum reorg, but "far outside" is not "impossible", and we would rather write it down than hide behind the word *finality*.

3. **We do not prevent liquidations.** We pay afterwards. Prevention requires Attestcoin Writability, outside this hackathon's scope.
4. **Flat 2% premium, not risk-priced.** Real pricing needs historical data and quantitative analysis.
5. **The underwriter side is simplified.** The vault is funded manually. There is no share accounting, lock period, or real underwriting economics yet.
6. **One chain, one protocol.** The architecture generalizes; the implementation does not yet.
7. **The receipt-decoding path is untested on a real network.** The precompile has no bytecode, so local tests stop at `MockProver`. This is the largest open risk remaining.

Roadmap

Four items, each born from a limitation above.

1. **Multiple source chains** (Arbitrum, Base, Polygon) — a single capital pool underwriting many chains is why Creditcoin was chosen in the first place.
2. **Capital efficiency** — underwriter capital sits idle for the whole policy period, and that opportunity cost becomes a price floor the user pays through premiums. Deploying idle capital into conservative yield strategies lowers that floor. Risk note: this adds new exposure, because an exploit in the destination protocol would erase claim funds exactly when they are most needed. The strategy therefore has to stay conservative and capped.
3. **Risk-priced premiums** — a flat rate is unfair to conservative positions and too cheap for aggressive ones.
4. **Automated protection via Writability** — preventing beats reimbursing.

Deployed contracts

Deployed 13 September 2026 from `0xf122a903448e0A43DC275897Df2e9E9FCCFe972c`.

CONTRACT	CHAIN	ADDRESS	DEPLOY TX
AniWereProbe	Sepolia	<code>0xa4a3eB202d89066909c5FDdEAdfDf79F a3da025a</code>	<code>0xcc83...a0e9</code>
AniWereASC	Creditcoin CC3	<code>0x1A5D249A8e711E2288AdD7c01e31Eb7F FB05D97E</code>	<code>0x8f79...c16e</code>
CoverVault	Creditcoin CC3	<code>0xBBCCd15B6bf49d7df3F9b7685455c25a 2b4e085e</code>	created by the ASC constructor, same tx
AttestcoinAdap ter	Creditcoin CC3	<code>0x45dd5B746490f93c90Ea5c21212c25C7 2160BEe5</code>	<code>0xc16e...ae14</code>
EvmV1Decoder (library)	Creditcoin CC3	<code>0xecd2a32b774f216f1b0be897077e3af9 c3502ff4</code>	<code>0x1aec...5ef4</code>

The vault was seeded with 500 CTC of underwriter capital via `depositCapital()` in `0x2e43...04c1`.

